

北京理工大学

Beijing Institute of Technology

Bachelor's Thesis

软件定义网络可观测全局控制平面的设计与
实现

Design and Implementation of an Observable
Global Control Plane for Software-Defined
Networks

School:	计算机学院
Degree:	计算机科学与技术
Author:	洪浩元
Student ID:	1820222026
Supervisor:	翟岩龙
Co-Supervisor:	Richard Ma (National University of Singapore)

May 12 2026

原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：

日期： 2026 年 5 月 1 日

关于使用授权的声明

本人完全了解北京理工大学有关保管、使用毕业设计（论文）的规定，其中包括：①学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；③学校可允许本毕业设计（论文）被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；⑤学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：

日期： 2026 年 5 月 1 日

指导老师签名：

日期： 2026 年 5 月 1 日

软件定义网络可观测全局控制平面的设计与实现

摘 要

互联网交换点（IXP）带宽转售商面临严峻挑战：客户需求随时间波动（白天 1Gbps、夜间 200Mbps），但传统合同分配固定容量，导致大量浪费或超额费用。本文设计并实现了一个分布式微服务控制平面，通过周期性密封拍卖机制动态分配 IXP 带宽，实现客户间的公平资源竞争和高效利用。系统由三个核心微服务组成：API 网关接受客户的带宽竞拍（指定路径与价格），拍卖运行器在固定间隔执行密封拍卖算法计算公平分配和清算价格，遥测处理器收集数据平面流量统计并提供反馈驱动客户下一轮竞拍决策。这些服务通过 Atomix 分布式一致性存储（Raft 协议）维护强一致的共享状态，通过 Apache Kafka 异步消息队列实现服务解耦与独立扩展。

微服务架构通过解耦实现扩展性，但引入了可观测性难题：“软故障”——控制平面逻辑破损（如共识层降级、消息处理延迟）但基础设施监控仍显示正常（容器运行、CPU 正常）。本文通过 OpenTelemetry 框架的综合可观测性设计解决此问题，对控制平面进行系统化仪器化以快速检测故障。框架采用指标（趋势检测）、分布式追踪（因果路径重建）、结构化日志（事件级证据）三支柱架构。关键技术是追踪上下文传播方案：通过在 Atomix 共享状态中嵌入上下文，保存了跨异步边界的 W3C TraceContext，实现了从竞拍提交、拍卖清算到带宽实施的端到端请求追踪。

实验评估通过受控故障场景（即网络流量突增期间的共识存储延迟降级）证明了框架的有效性。结果显示，可观测性框架能够实现快速的异常检测（警报在降级后数秒内触发），并能通过属性关联精准定位数据库瓶颈，使运维人员能够迅速区分网络拥塞与控制平面内部的系统故障。本系统为构建可观测的分布式微服务控制平面确立了工程实践，为设计任何需要强一致性和多服务协调的 SDN 应用提供了架构参考价值。

关键词：软件定义网络；可观测性；OpenTelemetry；分布式追踪；微服务；带宽拍卖

Design and Implementation of an Observable Global Control Plane for Software-Defined Networks

Abstract

Internet Exchange Point (IXP) bandwidth resellers face significant challenges due to fluctuating customer demand. Traditional contracts allocate fixed capacity, leading to massive waste during off-peak hours and potential packet loss during surges. This thesis designs and implements a distributed microservice control plane that dynamically allocates IXP bandwidth through a periodic sealed-bid auction mechanism, enabling fair resource competition and high utilization. The system consists of three core microservices: an API Gateway ingests customer bandwidth bids, an Auction Runner executes a uniform-price clearing algorithm at scheduled intervals, and a Telemetry Processor aggregates data-plane flow statistics to drive subsequent bidding decisions. These services maintain strongly consistent shared state via the Atomix distributed consensus store (Raft protocol) and achieve asynchronous decoupling via the Apache Kafka message broker.

While microservice decoupling provides scalability, it introduces observability challenges: “soft failures” occur when the control plane is logically broken (consensus degradation, message processing lag) yet infrastructure monitoring shows normalcy (containers healthy, CPU normal). This thesis addresses this gap through comprehensive observability design using the OpenTelemetry framework, systematically instrumenting the control plane to rapidly detect failures. The framework employs three complementary signals: metrics for trend detection, distributed traces for causal path reconstruction, and structured

Key Words: Software-Defined Networking; Observability; OpenTelemetry; Distributed Tracing; Microservices; Bandwidth Auction

Table of Contents

摘 要	I
Abstract	II
Chapter 1 Introduction	1
1.1 The IXP Bandwidth Resale Problem	1
1.2 SDN and Microservice Architectures	1
1.3 The Observability Gap in Asynchronous Systems	2
1.4 Terminology Definitions	2
1.5 Objectives and Contributions	3
1.6 Thesis Organization	3
Chapter 2 System Architecture	5
2.1 Architectural Overview	5
2.2 Asynchronous Decoupling via Kafka	6
2.3 Control Plane Workflows	8
2.3.1 The Auction System Workflow	8
2.3.2 The Network Telemetry Workflow	9
2.4 Summary	10
Chapter 3 System Implementation and Data Design	11
3.1 Distributed Consensus Storage Design	11
3.1.1 Atomix Cluster Configuration	11
3.2 Core Data Models and Schema Design	12
3.2.1 Bid Entity Schema	13
3.2.2 Allocation and Credit History Schemas	14
3.3 Inter-Service Communication Protocols	15
3.3.1 Data Plane Telemetry Contracts	15
3.4 Network Telemetry Processing Pipeline	16
3.4.1 State Aggregation and Rate Calculation	16
3.4.2 Flow State Persistence	16
3.5 Auction Clearing Implementation	16
3.5.1 The Uniform-Price Mathematical Formulation	17
3.5.2 Reservation Price Extension via Virtual Bidding	18

3.6	Test Environment Simulation	19
3.6.1	The Simulated Agent (Bid Simulator)	19
3.6.2	The Simulated Switch (Telemetry Simulator)	19
3.7	Summary	20
Chapter 4	Observability Framework Design	21
4.1	Design Principles	21
4.2	The Unified Telemetry Pipeline	21
4.2.1	Metrics (Prometheus)	22
4.2.2	Structured Logs (Loki)	22
4.2.3	Distributed Tracing (Jaeger)	22
4.3	Asynchronous Trace Context Propagation	22
4.3.1	Context Injection in the Gateway	23
4.3.2	Context Extraction and Linking in the Runner	23
4.4	Telemetry Data Reduction Strategy	23
4.4.1	Tail-Based Trace Sampling	23
4.4.2	Severity-Based Log Filtering	24
4.5	Summary	24
Chapter 5	System Evaluation	25
5.1	Experimental Setup	25
5.2	Baseline Validation: System Health and Traceability	26
5.2.1	API Throughput and State Consistency	26
5.2.2	Verification of Asynchronous Trace Propagation	26
5.2.3	Auction Execution Latency	27
5.3	Degradation Scenario: Observability Under Silent Failure	28
5.3.1	Scenario Setup and Fault Model	28
5.3.2	Expected Observability Behavior	28
5.3.3	Degradation Scenario Results	29
5.3.4	Observability Value Proposition	32
5.4	Summary	33
Chapter 6	Conclusions	34
6.1	Summary of Contributions	34
6.1.1	Decoupled SDN Control Plane Design	34
6.1.2	Asynchronous Observability Framework	34

6.1.3	System Validation and Fault Localization	35
6.2	Future Directions	35
6.2.1	Machine Learning-Driven Anomaly Detection	35
6.2.2	Federated Multi-Region Control Planes	35
6.2.3	Dynamic Economic Pricing Mechanisms	35
	References	37
	Acknowledgements	39

Chapter 1 Introduction

1.1 The IXP Bandwidth Resale Problem

Internet Exchange Points (IXPs) are critical infrastructure where networks peer to exchange traffic directly^[1-2]. Large enterprises and content delivery networks operate massive networks with global presence, while smaller networks and emerging providers lack the scale to negotiate peering directly with these giants. The traditional solution is to purchase “transit” bandwidth—a small network contracts with a transit provider for a fixed committed amount of bandwidth, such as 1 Gbps, under a long-term agreement.

However, this model introduces significant inefficiency. Bandwidth demand varies throughout the day: a small network might need 1 Gbps during peak hours but only 200 Mbps during off-peak periods, resulting in 80% capacity waste during low-demand windows. Conversely, unexpected traffic spikes can exceed contracted capacity, forcing expensive emergency purchases or resulting in severe packet loss. This demand volatility, driven by diurnal traffic patterns and unpredictable network events, creates a highly lucrative market opportunity for dynamic bandwidth resale.

A transit reseller purchases bulk bandwidth from major providers (e.g., 100 Gbps from major carriers) and subdivides it among many smaller customers. If this subdivision is static, the reseller faces the same inefficiencies as the customers. To maximize link utilization and profitability, the reseller requires a dynamic, competitive allocation mechanism. This thesis addresses this need by designing a Software-Defined Networking (SDN) global control plane that treats network bandwidth as a dynamic commodity, allocated via periodic economic auctions.

1.2 SDN and Microservice Architectures

To facilitate a global, high-throughput bandwidth marketplace, the control plane must be highly scalable and resilient. Traditional monolithic SDN controllers^[3] create single points of failure and struggle to scale individual logical components (such as bid ingestion versus flow metric processing).

Modern cloud-native design dictates that the control plane be decomposed into isolated

microservices^[4]. By utilizing a state-separated architecture—where stateless microservices communicate via an asynchronous message broker (Apache Kafka)^[5] and rely on a distributed consensus store (Atomix)^[6] for a single source of truth—the system achieves both horizontal scalability and strict fault isolation.

1.3 The Observability Gap in Asynchronous Systems

While asynchronous microservices provide superior scalability, they introduce severe operational blind spots. In a traditional synchronous system, a failing component crashes, and standard infrastructure orchestrators (like Kubernetes) simply restart it.

However, state-separated SDN controllers frequently suffer from “soft failures.” For example, if the Atomix consensus database experiences severe network latency, the surrounding microservices do not crash; they simply hang, waiting for database locks. Standard infrastructure monitoring tools^[7] (e.g., CPU/Memory metrics and LivenessProbes) report the system as perfectly healthy, while the actual business logic (the auction clearing) is silently paralyzed.

Furthermore, because communication is asynchronous, traditional request tracing fails^[8]. When an API Gateway writes a bid to a database, and an Auction Runner reads it 60 seconds later, the causal link between the customer's HTTP request and the auction algorithm's execution is lost. Bridging this observability gap in asynchronous environments is the primary technical challenge addressed by this thesis^[9].

1.4 Terminology Definitions

Because this thesis spans both network engineering and distributed software engineering, it is critical to explicitly differentiate between two distinct types of telemetry utilized within the system architecture:

- **Network Telemetry (Business Logic):** Refers to data-plane traffic statistics (e.g., bytes transferred, flow rates in Kbps) periodically emitted by the physical or simulated switches. This data is processed by the control plane and served to customer agents to inform their economic bidding strategies.
- **Observability Telemetry (System Health):** Refers to the operational signals (traces,

logs, and infrastructure metrics) generated by the OpenTelemetry SDKs embedded within the Go microservices. This data is used exclusively by system operators to monitor the software's health, debug latency, and detect application-layer failures.

1.5 Objectives and Contributions

The primary objective of this thesis is to design, implement, and evaluate a microservice-based SDN global control plane that is both economically efficient and fully observable. The core contributions are as follows:

1. **State-Separated Auction Control Plane:** The design and implementation of a microservice architecture that dynamically shapes network traffic based on a periodic, sealed-bid uniform-price auction algorithm.
2. **Asynchronous Trace Context Propagation:** The development of a custom W3C TraceContext propagation strategy that utilizes the Atomix consensus database as a context carrier, enabling unbroken distributed tracing across temporally decoupled services.
3. **Observability-Driven Fault Localization:** The integration of a comprehensive OpenTelemetry framework (Metrics, Traces, and Logs) to explicitly detect and localize soft failures (consensus degradation) that evade traditional container orchestration monitoring.

1.6 Thesis Organization

The remainder of this thesis is structured as follows:

- **Chapter 2 (System Architecture):** Details the high-level microservice topology, the asynchronous decoupling strategy using Apache Kafka, and the primary data workflows.
- **Chapter 3 (System Implementation):** Provides an in-depth analysis of the control plane's core logic, including the mathematical formulation of the uniform-price auction and the custom trace-linking code.

- **Chapter 4 (Observability Framework Design):** Presents the OpenTelemetry architecture, detailing the data reduction strategies (tail-sampling and severity-filtering) used to minimize performance overhead.
- **Chapter 5 (System Evaluation):** Evaluates the system's functional correctness during a massive traffic spike, and validates the observability framework's ability to detect an injected database latency fault.
- **Chapter 6 (Conclusions):** Summarizes the contributions and proposes directions for future research.

Chapter 2 System Architecture

This chapter presents the complete system architecture for the SDN global control plane. It details the microservice layout, the asynchronous decoupling strategy, and the specific data workflows that enable the dynamic bandwidth marketplace. By moving away from monolithic controller designs, this architecture ensures high availability and scalability, though it introduces significant distributed tracking challenges that necessitate a custom observability framework.

2.1 Architectural Overview

The complete architecture of the SDN global control plane is illustrated in Figure 2-1. The system is organized into three primary microservices: customer request handling (API Gateway), resource allocation (Auction Runner), and measurement aggregation (Telemetry Processor).

As shown in the figure, these services do not communicate directly via synchronous remote procedure calls (RPCs). Instead, they interact through a state-separated architecture utilizing a shared consensus store (Atomix) and an asynchronous message broker (Apache Kafka).

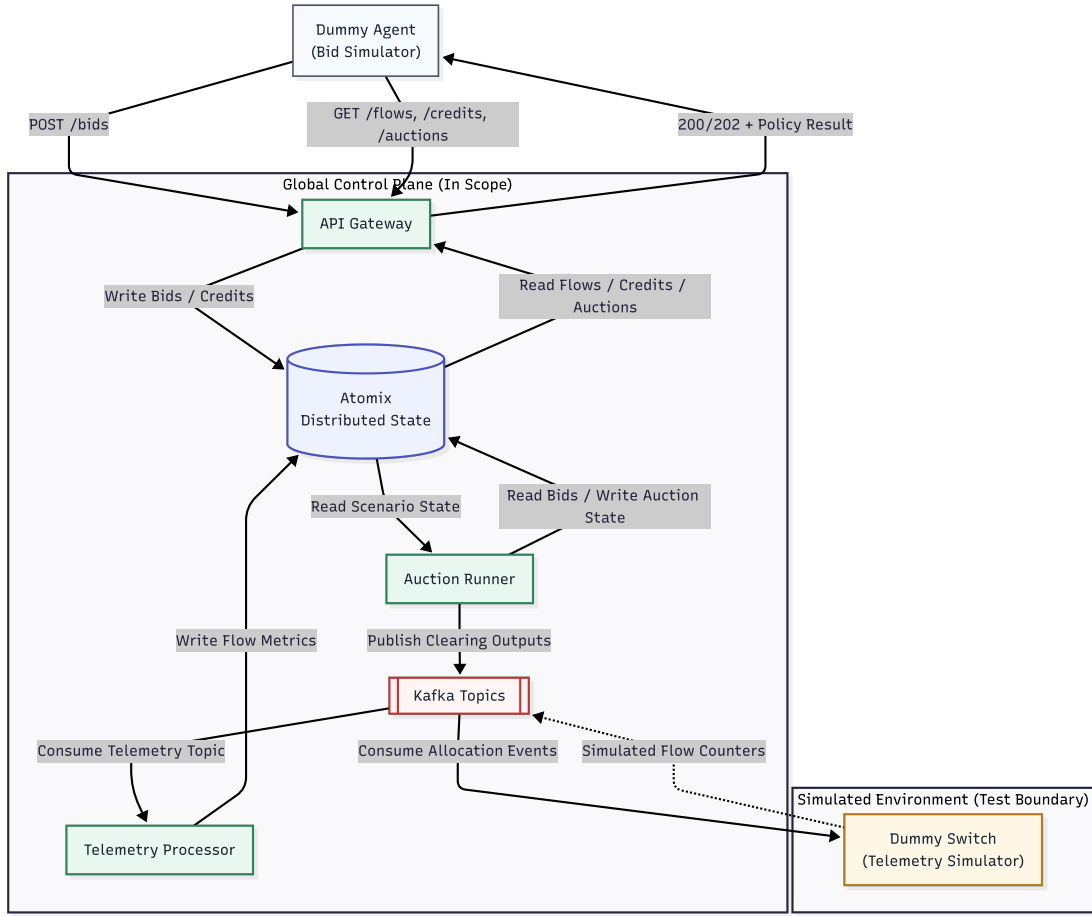


Figure 2-1 System architecture of the observable SDN global control plane, illustrating the separation of state and asynchronous event routing.

Customer bids flow into the API Gateway and are written directly to Atomix, ensuring immediate consistency across replicas. At scheduled intervals, the Auction Runner pulls these bids, computes allocations, and pushes results to Kafka. Finally, the simulated data-plane agents consume these allocations while simultaneously publishing flow statistics back to Kafka, which the Telemetry Processor ingests.

2.2 Asynchronous Decoupling via Kafka

To prevent slow physical network hardware from stalling the control plane's economic logic, the system utilizes Apache Kafka^[5] as a strict decoupling boundary, as illustrated in Figure 2-2.

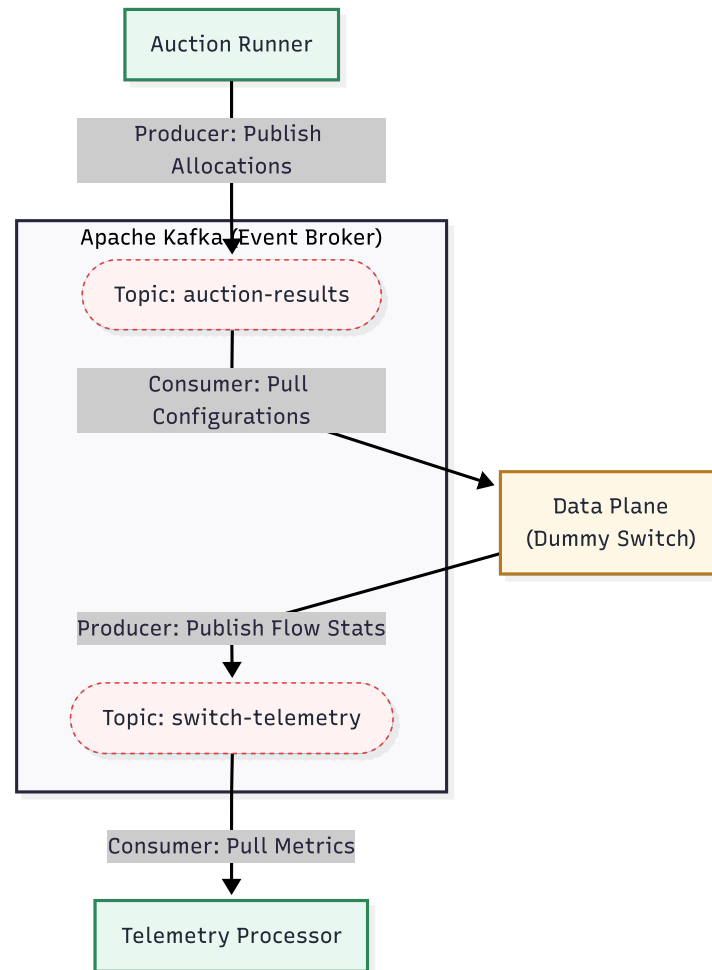


Figure 2-2 The Publisher/Subscriber decoupling model utilizing Apache Kafka for asynchronous event distribution.

When the Auction Runner completes its clearing algorithm, it publishes final allocations to the `auction-results` topic. The runner does not wait for the switches to apply these configurations. The data plane agents (represented in testing by the Simulated Switch) act as Consumers, pulling these allocations at their own pace. Conversely, the switches continuously publish high-frequency network metrics to the `switch-telemetry` topic, which the Telemetry Processor consumes. This publisher/subscriber model isolates failure domains, ensuring that localized data plane degradation does not instantly crash the global control plane.

2.3 Control Plane Workflows

Within this decoupled architecture, the system executes two primary business workflows: the Auction System and the Network Telemetry System. The temporal separation inherent in these workflows is the primary driver for the advanced trace-linking implementation discussed later in this thesis.

2.3.1 The Auction System Workflow

The Auction System handles the ingestion of customer bids and the execution of the economic clearing algorithm. The lifecycle of a bandwidth request is illustrated in Figure 2-3.

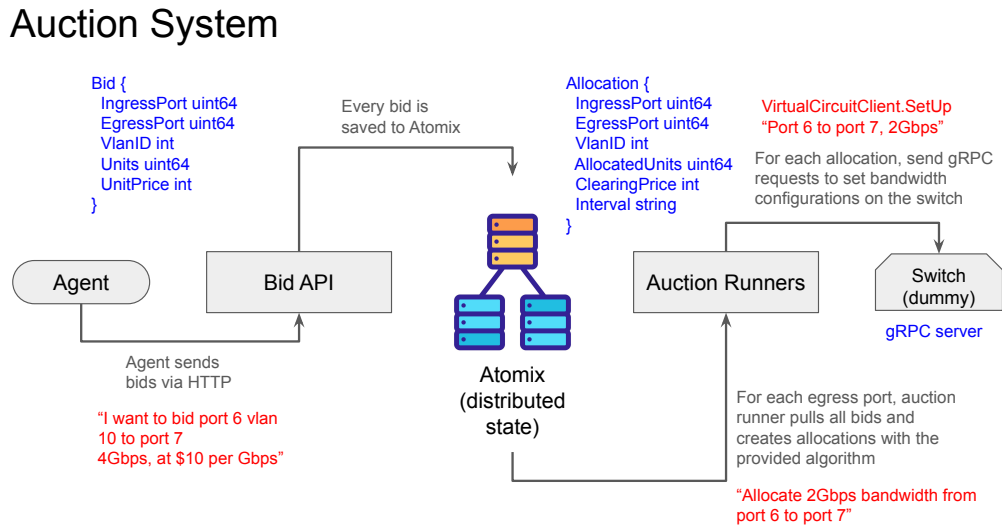


Figure 2-3 The Auction System workflow, detailing the path from bid submission to data plane allocation.

Customer agents interface with the API Gateway via HTTP REST endpoints^[10]. A customer submits a Bid payload explicitly defining their IngressPort, EgressPort, VlanID, requested Units (in Kbps), and their maximum UnitPrice.

Upon receiving a valid request, the API Gateway immediately writes this bid into Atomix. At the scheduled interval, the Auction Runner pulls all accumulated bids for an egress port, executes the uniform-price allocation algorithm, and generates an Allocation record. This

record is published to Kafka, where the dummy switch consumes it and simulates data plane enforcement via gRPC (`VirtualCircuitClient.SetUp`).

2.3.2 The Network Telemetry Workflow

To facilitate a dynamic and competitive marketplace, customer agents must be able to base their bids on real-time network conditions. The Network Telemetry System processes data-plane flow statistics for this purpose, as shown in Figure 2-4.

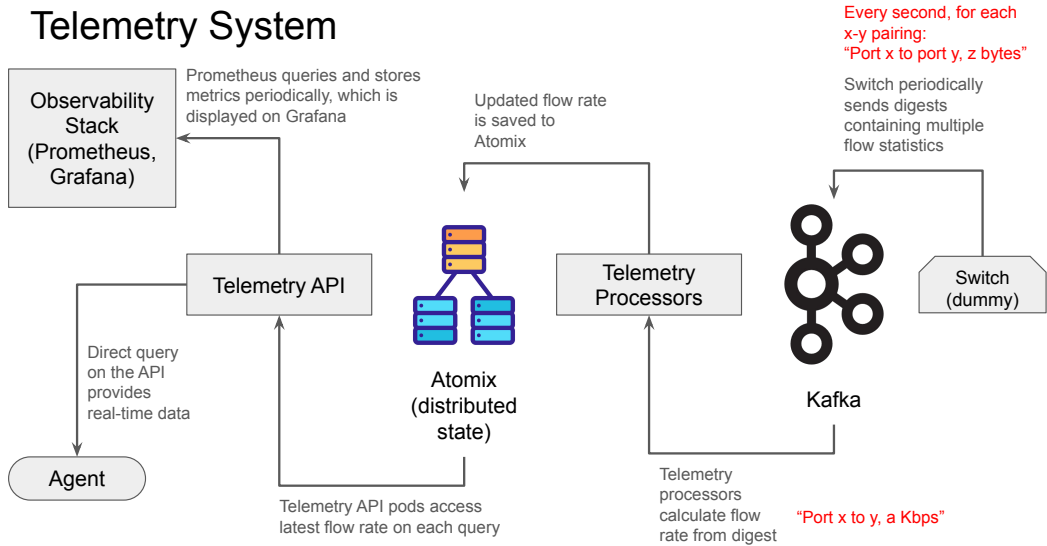


Figure 2-4 The Network Telemetry workflow, illustrating how raw switch statistics are processed and served to bidding agents.

Every second, the Simulated Switch emits a payload containing absolute transfer counts and publishes it to the `switch-telemetry` topic. The Telemetry Processor microservices continuously consume these counts, perform windowed calculations to convert them into standardized flow rates (Kbps), and update the corresponding network state records within Atomix. When a Customer Agent prepares to bid, it executes an HTTP GET query against the API Gateway's Telemetry API to retrieve this real-time snapshot of network congestion.

2.4 Summary

This chapter presented the complete system architecture and data workflows for the SDN global control plane. The microservice design utilizes Atomix for consistent state management and Kafka for asynchronous hardware decoupling. The detailed Auction and Telemetry workflows demonstrate exactly how customer intent is transformed into network reality.

However, because these workflows are temporally decoupled—where a bid is written by one service and read by another service a full minute later—traditional monitoring tools lose track of the request lifecycle. The next chapter explores the specific implementation logic behind these workflows, providing the foundation necessary to understand how the custom observability framework solves this asynchronous tracking problem.

Chapter 3 System Implementation and Data Design

This chapter details the concrete engineering implementation of the SDN control plane. To transition the high-level architecture into a functional distributed system, careful consideration must be given to data persistence, inter-service communication contracts, and core algorithmic logic. This chapter unpacks the Atomix database deployment, the internal entity-relationship models, the Protocol Buffer telemetry pipeline, the mathematical implementation of the auction logic, and the simulated boundary components.

3.1 Distributed Consensus Storage Design

The control plane relies entirely on Atomix as its single source of truth. Because multiple microservices (API Gateway, Auction Runner, Telemetry Processor) concurrently read and write to this state, a standard relational database is insufficient. The system implements a distributed consensus store based on the Raft consensus algorithm to guarantee linearizable reads and writes across the cluster.

3.1.1 Atomix Cluster Configuration

To ensure high availability and partition tolerance, the Atomix cluster is deployed natively within Kubernetes using a Custom Resource Definition (CRD). The configuration explicitly provisions a multi-replica Raft group.

As defined in the system's `store.yaml` configuration (Figure 3-1), the `ConsensusStore` is configured with `replicas: 3` and `groups: 3`. This specific replication factor ensures that the cluster can tolerate the failure of a minority node while maintaining a strict quorum for leader election and log replication. Furthermore, the storage class dynamically provisions 8Gi of persistent block storage per node to ensure transaction logs survive container restarts.

```
apiVersion: consensus.atomix.io/v1beta1
kind: ConsensusStore
metadata:
  name: consensus-store
spec:
  replicas: 3
  groups: 3
  volumeClaimTemplate:
    spec:
      accessModes:
        - ReadWriteOnce
      volumeMode: Filesystem
      resources:
        requests:
          storage: 8Gi
      storageClassName: standard
```

Figure 3-1 The Kubernetes deployment configuration for the Atomix Consensus Store, specifying a 3-node fault-tolerant Raft architecture.

The microservices bind to this cluster using a `StorageProfile`, which abstracts the physical pod IP addresses into a logical routing layer, allowing the Go microservices to connect seamlessly via gRPC.

3.2 Core Data Models and Schema Design

To support the dynamic auction marketplace, the Atomix key-value store manages several core data structures. Because Atomix operates as a NoSQL document store, these entities are defined as serialized Go structs and stored as byte arrays. The relationships between these entities are illustrated in Figure 3-2.

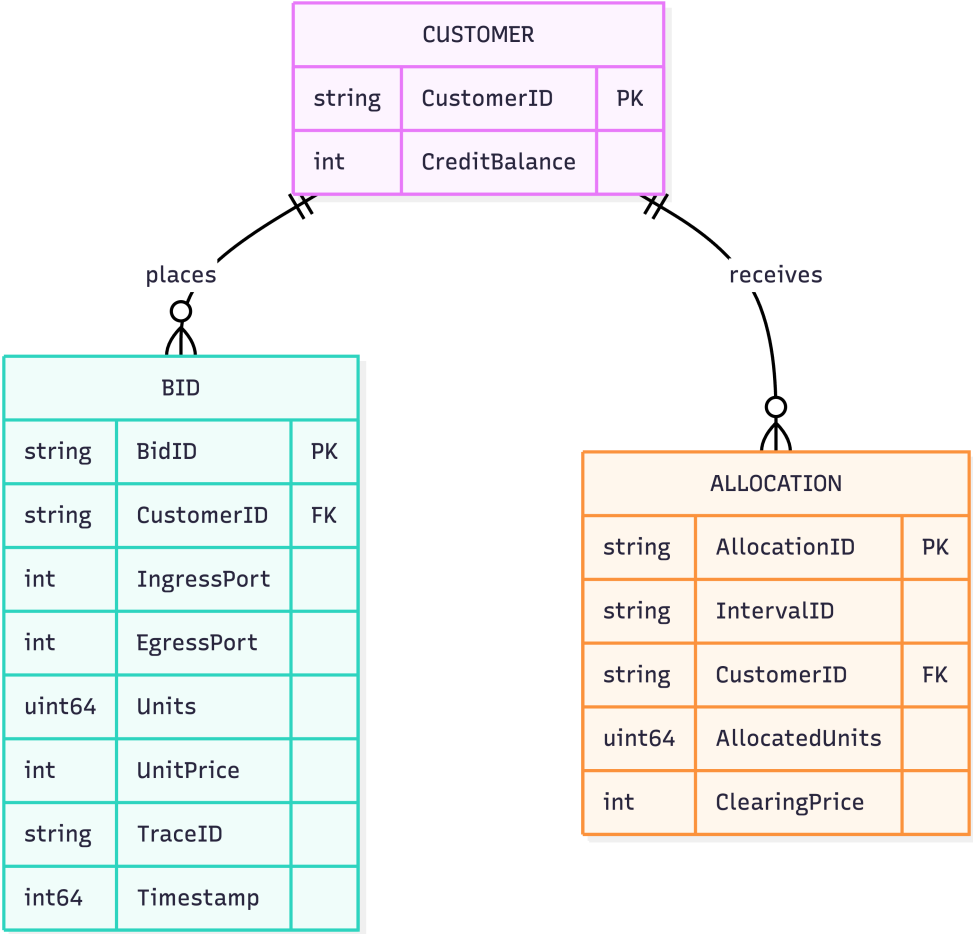


Figure 3-2 Entity-Relationship diagram of the core control plane data structures.

3.2.1 Bid Entity Schema

The Bid entity represents a customer’s intent to purchase bandwidth for a specific network path. The API Gateway validates incoming HTTP requests and constructs this entity before committing it to Atomix. Table 3-1 details the schema fields.

Table 3-1 Database Schema: Bid Entity

Field Name	Data Type	Description
BidID	String (UUID)	Primary Key. Uniquely identifies the bid transaction.
CustomerID	String	Foreign Key linking to the originating AS agent.
IngressPort	Integer	The network entry point for the traffic flow.
EgressPort	Integer	The congested bottleneck port being auctioned.
Units	Unsigned Int64	Requested bandwidth volume in Kbps.
UnitPrice	Integer	Maximum willingness to pay per unit of bandwidth.
TraceID	String	Injected OpenTelemetry trace context carrier.
Timestamp	Epoch (Int64)	Unix timestamp of ingestion for interval filtering.

3.2.2 Allocation and Credit History Schemas

Once the Auction Runner computes the clearing algorithm, it generates Allocation records and updates the customer's Credit profile. Tracking credit ensures that agents operate within a finite budget, preventing malicious actors from continuously submitting infinite bids to monopolize the network. Table 3-2 details the Allocation schema.

Table 3-2 Database Schema: Allocation Entity

Field Name	Data Type	Description
AllocationID	String (UUID)	Primary Key for the granted bandwidth reservation.
IntervalID	String	Links the allocation to a specific 30s auction round.
CustomerID	String	Identifies the recipient of the bandwidth.
AllocatedUnits	Unsigned Int64	Final granted bandwidth (may be less than requested).
ClearingPrice	Integer	The uniform market price applied to the allocated units.

The system maintains an AuctionHistory ledger for each customer. At the end of every interval, the total cost ($\text{AllocatedUnits} \times \text{ClearingPrice}$) is deducted from the customer's active credit balance. If an agent exhausts its credits, the API Gateway rejects its future bids with an HTTP 402 Payment Required status.

3.3 Inter-Service Communication Protocols

In a decoupled microservice architecture, the choice of data serialization format significantly impacts system latency and network throughput. While the API Gateway utilizes standard JSON over HTTP for external customer interactions, internal asynchronous communication via Apache Kafka requires a highly optimized binary format.

To satisfy this requirement, the control plane utilizes Google Protocol Buffers (Protobuf) for all internal message passing. Protobuf provides a strictly typed, language-agnostic contract between services, ensuring backward compatibility and significantly smaller payload sizes compared to JSON.

3.3.1 Data Plane Telemetry Contracts

The most high-frequency communication path in the system is the telemetry pipeline, where the simulated data plane continuously streams network statistics back to the control plane. This contract is defined by the `flow.proto` schema.

As detailed in Table 3-3, the base unit of network measurement is the `FlowDigest`. Because physical network switches operate at line rate, they do not inherently understand higher-level metrics like “Kilobits per second” (Kbps). Instead, they maintain hardware counters of absolute bytes transferred for a given ingress-egress port pair. The `TelemetryReport` wraps multiple flow digests into a single timestamped batch to minimize Kafka producer overhead.

Table 3-3 Protobuf Schema: Data Plane Telemetry Report (`flow.proto`)

Message	Field	Description
FlowDigest	ingress_port	(uint64) The physical entry port of the traffic.
	egress_port	(uint64) The physical exit port of the traffic.
	bytes_transferred	(uint64) Absolute counter of bytes routed.
	packets_dropped	(uint64) Counter of packets dropped due to congestion.
TelemetryReport	switch_id	(string) Unique identifier of the reporting hardware.
	timestamp	(int64) Unix epoch time of the hardware snapshot.
	flows	(repeated FlowDigest) Array of flow counters.

3.4 Network Telemetry Processing Pipeline

While the Auction Runner handles the economic allocation of bandwidth, the marketplace cannot function unless bidding agents have access to real-time network congestion data. The Telemetry Processing Pipeline is responsible for transforming the raw, absolute hardware counters into actionable, relative flow rates.

3.4.1 State Aggregation and Rate Calculation

The Telemetry Processor microservice operates as a continuous Kafka consumer, subscribed to the `switch-telemetry` topic. Upon ingesting a `TelemetryReport` protobuf message, the processor deserializes the byte payload and begins the aggregation phase.

Because the switch only provides absolute byte counters, the processor must maintain state to calculate the current flow rate. The system queries the Atomix consensus store to retrieve the previous byte count and timestamp for a specific flow. The real-time throughput is then derived using the following rate equation:

$$\text{Throughput (Kbps)} = \frac{(\text{CurrentBytes} - \text{PreviousBytes}) \times 8}{(\text{CurrentTime} - \text{PreviousTime})} \times \frac{1}{1000} \quad (3-1)$$

3.4.2 Flow State Persistence

Once the relative throughput and packet drop rates are calculated, the Telemetry Processor marshals this derived data back into a structured Go entity (`FlowMetrics`). This entity is subsequently committed back into the Atomix consensus store under a hierarchical routing key format. By persisting this derived state into Atomix, the architecture achieves a highly efficient read-path. When a Customer Agent prepares to formulate a bid, it performs a direct, low-latency read from the Atomix key-value store, bypassing the heavy computational overhead of the Kafka streams.

3.5 Auction Clearing Implementation

The Auction Runner acts as the economic engine of the system. Rather than reacting to customer requests synchronously, it utilizes an interval-triggered batch processing model as illustrated in (Figure 3-3) to compute fair bandwidth allocations, querying the Atomix state

store to process bids collectively.

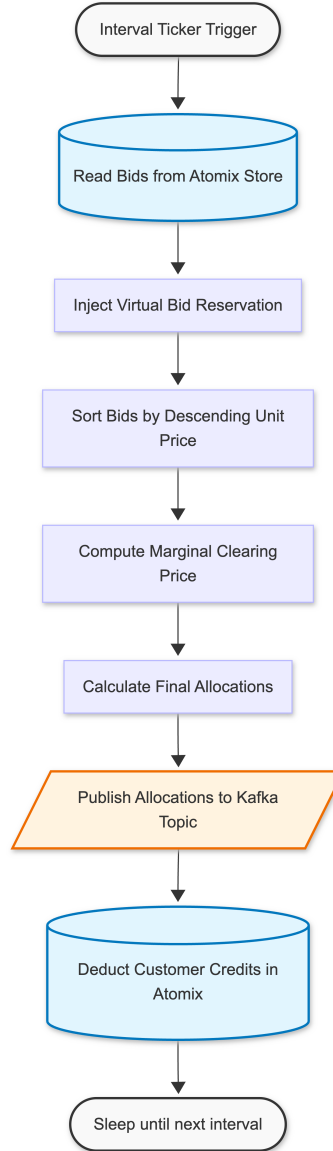


Figure 3-3 Execution flow of the Auction Runner, detailing the interval trigger, state retrieval, clearing computation, and asynchronous result publication.

3.5.1 The Uniform-Price Mathematical Formulation

At each interval trigger, the runner executes a sealed-bid, uniform-price auction. Let the total available bandwidth capacity for a specific egress port be denoted as C . The system retrieves a set of n eligible bids $B = \{b_1, b_2, \dots, b_n\}$ from Atomix. Each bid b_i is represented as a tuple (q_i, p_i) , where q_i denotes the requested bandwidth quantity (Units) and p_i

represents the maximum willingness to pay (UnitPrice).

To execute the auction, the algorithm first sorts the bids in descending order of price, such that $p_1 \geq p_2 \geq \dots \geq p_n$.

Clearing Price Determination: The algorithm iterates through the sorted bids, accumulating the demanded quantity until the capacity C is exhausted. The clearing price P^* is defined by the price of the marginal bid k where the cumulative demand meets or exceeds capacity:

$$P^* = p_k \quad \text{where} \quad \sum_{i=1}^{k-1} q_i < C \quad \text{and} \quad \sum_{i=1}^k q_i \geq C \quad (3-2)$$

If the total demand is strictly less than the capacity ($\sum q_i < C$), the clearing price defaults to the price of the lowest submitted bid ($P^* = p_n$).

Allocation Rule: Let x_i denote the final bandwidth allocated to bid b_i . The allocation is divided into three tiers based on the clearing price:

1. **Inframarginal Bids** ($p_i > P^*$): These bids clear the market and are fully allocated.

$$x_i = q_i \quad (3-3)$$

2. **Extramarginal Bids** ($p_i < P^*$): These bids fail to clear the market and are rejected.

$$x_i = 0 \quad (3-4)$$

3. **Marginal Bids** ($p_i = P^*$): These bids tie exactly at the clearing price. Let $M = \{j \mid p_j = P^*\}$ be the set of marginal bids, and $D_M = \sum_{j \in M} q_j$ be their total demand. The remaining capacity available for these marginal bids is $C_{rem} = C - \sum_{i: p_i > P^*} q_i$. Marginal bids receive a strictly proportional share of the remaining capacity:

$$x_i = \min \left(q_i, \quad q_i \times \frac{C_{rem}}{D_M} \right) \quad \forall i \in M \quad (3-5)$$

3.5.2 Reservation Price Extension via Virtual Bidding

To prevent bandwidth from being sold at unsustainably low prices during periods of low demand, the algorithm must support a strict Reservation Price (R).

Rather than introducing complex branching logic into the core algorithm, the reservation price is elegantly implemented via a *Virtual Bid injection*. Before the sorting phase, the Go implementation automatically generates a virtual bid $b_v = (C, R)$ —requesting the entire link capacity at the minimum reservation price—and appends it to the bid set B .

The standard uniform-price sorting and clearing equations are then executed on this augmented set $B \cup \{b_v\}$. This mathematically guarantees that the clearing price P^* is strictly bounded: $P^* \geq R$. Crucially, during the proportional allocation phase for marginal bids (if $P^* = R$), the algorithm excludes the virtual bid from the marginal demand calculation (D_M), ensuring that the virtual bid successfully establishes a price floor without artificially diluting the proportional allocation share of genuine customer bids.

3.6 Test Environment Simulation

To rigorously test the mathematical clearing algorithm and the telemetry processing pipeline without the overhead or latency of physical network hardware, the system implements simulated boundary components written in Go.

3.6.1 The Simulated Agent (Bid Simulator)

The Simulated Agent acts as the upstream customer. It is implemented as a high-throughput load generator capable of reading from configuration-driven scenario files (YAML) to simulate various market conditions, such as diurnal traffic spikes or sudden port congestion. By continuously firing HTTP POST requests at the API Gateway, it stresses the Atomix write-throughput and validates the robustness of the auction algorithm under heavy concurrent load.

3.6.2 The Simulated Switch (Telemetry Simulator)

The Simulated Switch acts as the downstream data plane. It connects directly to the Kafka cluster to consume the `final auction-results` messages generated by the algorithm. Simultaneously, it runs a background goroutine that generates simulated flow digests (representing hardware byte counters) and publishes them to the `switch-telemetry` topic every second. This isolates the control plane logic, ensuring that any evaluated performance benchmarks reflect software efficiency rather than physical switch hardware limitations.

3.7 Summary

This chapter detailed the logical implementation and data design of the control plane's core functionalities. By explicitly defining the Atomix consensus storage, the protobuf contracts, and the mathematical telemetry processing, the system is proven to be robust and highly scalable. The auction algorithm utilizes formal mathematical uniform-pricing to ensure fair bandwidth allocation, successfully enhanced by a virtual-bid software mechanism to enforce reservation pricing.

While the economic logic and data pipelines are now firmly established, operating these asynchronous microservices in production requires deep systemic visibility. The next chapter presents the Observability Framework, detailing how the system is instrumented to track these complex algorithms and expose distributed failures.

Chapter 4 Observability Framework Design

This chapter presents the observability framework designed to expose logical failures within the asynchronous SDN control plane. Rather than relying on generic infrastructure metrics (which fail to capture decoupled architectural bottlenecks), this framework directly instruments the business logic and service dependencies. A major technical contribution of this chapter is the implementation of a custom trace-context propagation strategy that preserves request visibility across the Atomix consensus database.

4.1 Design Principles

The observability framework follows three core design principles that guide the implementation choices throughout the microservice environment.

1. **Business-Level Visibility:** Instrumentation must expose control-plane decision outcomes, not just container health. Metrics include auction execution latency, clearing prices, and bandwidth allocations. This ensures operators monitor the actual economic network shaping, rather than just CPU utilization.
2. **Low Operational Coupling:** Observability infrastructure must fail independently of request-critical code paths. If a tracing backend crashes, it must not cascade and block the Auction Runner from computing bandwidth allocations.
3. **Actionable Signal Correlation:** Metrics detect anomalies, traces localize the bottleneck, and structured logs verify the root cause. These signals must share common identifying attributes (e.g., `EgressPort`, `IntervalID`) to allow operators to pivot seamlessly between tools during an incident.

4.2 The Unified Telemetry Pipeline

To achieve these principles without vendor lock-in, the system utilizes the open-source OpenTelemetry (OTel) framework^[11-12]. The Go microservices are instrumented with the OTel SDKs^[13], pushing telemetry data to a centralized OpenTelemetry Collector^[14] deployed within the Kubernetes cluster.

4.2.1 Metrics (Prometheus)

Metrics provide real-time aggregation of system state^[15] and are used to trigger alerts. The API Gateway exposes HTTP throughput and error rates, while the Auction Runner exposes the uniform-price auction durations and the final clearing prices. These time-series metrics are scraped by Prometheus and visualized in Grafana dashboards^[16], establishing the baseline health of the SDN marketplace.

4.2.2 Structured Logs (Loki)

Traditional unstructured text logs are notoriously difficult to query in distributed systems. The framework mandates structured JSON logging. Every log emitted by the Go services is enriched with contextual attributes (such as `CustomerID` or `TraceID`) and forwarded to Grafana Loki. This transforms generic error strings into highly indexable, queryable events.

4.2.3 Distributed Tracing (Jaeger)

Distributed tracing^[8,17] reconstructs the causal chain of a request as it travels across the network. By generating a unique W3C `TraceID`^[18] at the API Gateway, the system attempts to track the entire lifecycle of a bandwidth bid. This data is exported to Jaeger for visualization. However, because the control plane uses an asynchronous database-mediated architecture, standard HTTP trace propagation is insufficient.

4.3 Asynchronous Trace Context Propagation

A fundamental limitation of standard distributed tracing is its reliance on synchronous HTTP/gRPC headers to pass the `TraceID` from one service to another. In this architecture, the API Gateway and the Auction Runner never communicate directly. The Gateway writes a bid to Atomix, and the Runner reads it from Atomix up to 60 seconds later. Standard tracing completely loses the context at the database boundary.

To solve this, the framework implements a custom database-carrier pattern to bridge the asynchronous gap.

4.3.1 Context Injection in the Gateway

When a customer submits a bid, the API Gateway initiates an OpenTelemetry root trace. Before marshaling the bid payload to be saved into the Atomix consensus store, the Gateway's Go handler explicitly extracts the current W3C TraceID and SpanID^[19] from the active context. It injects these exact values as string fields directly into the Bid data structure. Consequently, the Atomix database record itself acts as the trace context carrier.

4.3.2 Context Extraction and Linking in the Runner

When the Auction Runner wakes up to process an interval, it initiates a completely new root trace for the batch job. As it iterates through the bids pulled from Atomix, it extracts the stored TraceID and SpanID strings from the retrieved payloads.

Instead of trying to force the runner's execution to become a "child" of a 60-second-old HTTP request, the runner uses the OpenTelemetry Go SDK to create a Link. This explicit linking explicitly connects the runner's current execution span with the API Gateway's original bid submission span. This programmatic reconstruction successfully bridges the asynchronous boundary, proving that a specific auction batch job successfully cleared a specific customer's HTTP request.

4.4 Telemetry Data Reduction Strategy

Instrumenting every function across a high-throughput control plane generates a massive volume of telemetry data, potentially consuming more network bandwidth and storage than the control plane itself. The OpenTelemetry Collector mitigates this through advanced data reduction strategies.

4.4.1 Tail-Based Trace Sampling

Instead of sampling traces randomly at the edge (Head-Based Sampling), the framework implements Tail-Based Sampling^[20] at the Collector. The Collector buffers all trace spans in memory until the transaction completes. If the trace contains an error, a timeout, or an abnormally high latency, 100% of the trace is retained and exported to Jaeger. If the trace is fast and successful, it is aggressively down-sampled (e.g., keeping only 5% of healthy traces). This guarantees that operators never miss a failing transaction while drastically

reducing storage overhead^[21].

4.4.2 Severity-Based Log Filtering

Similarly, the Collector^[22] acts as a filter for structured logs. Debug and Info-level logs are dropped at the edge during normal operation, while Warning and Error-level logs are prioritized and shipped to Loki^[23].

4.5 Summary

This chapter detailed the design and implementation of the control plane's observability framework. By leveraging OpenTelemetry, the architecture successfully integrates metrics, structured logs, and distributed traces without tightly coupling to specific vendors.

Crucially, the implementation of custom context-carrier injection solves the tracking problem inherent in decoupled database architectures, ensuring continuous request visibility from bid submission to data plane enforcement. Furthermore, data reduction strategies at the Collector ensure this visibility incurs negligible performance overhead. The next chapter evaluates this framework in action, demonstrating its capability to detect and localize a severe infrastructure fault.

Chapter 5 System Evaluation

This chapter evaluates the performance, correctness, and observability of the SDN global control plane. To ensure clarity, the evaluation is divided into two phases. First, we establish a baseline of functional correctness by verifying the system's internal telemetry under normal load. Second, we demonstrate how the observability framework detects and isolates a distributed soft failure injected during runtime^[8,17,24].

5.1 Experimental Setup

To ensure reproducible results, the evaluation was conducted using an automated, configuration-driven Kubernetes testbed, with the key parameters summarized in Table 5-1.

The testbed abstracts physical hardware by utilizing a Simulated Switch to simulate data-plane enforcement. Traffic demand is generated by a pool of simulated Customer Agents. For this evaluation, all agents utilize a “Conservative” bidding strategy, generating a steady, predictable volume of bandwidth requests. This isolates the performance of the control plane's core uniform-price algorithm from advanced market dynamics.

The physical link capacity for the simulated egress port was constrained to 100 Kbps. The unified observability stack (Prometheus, Jaeger, Loki, OpenTelemetry Collector) was deployed alongside the core microservices to monitor system health in real-time.

Table 5-1 Baseline Experimental Configuration Parameters

Parameter	Value
Auction Interval	30 seconds
Telemetry Emission Interval	1 second
Data Plane Egress Capacity	100 Kbps (Simulated)
Minimum Reservation Price	50
Agent Bidding Strategy	Conservative (Deterministic)

5.2 Baseline Validation: System Health and Traceability

Before evaluating the system's fault-detection capabilities, it is necessary to establish a baseline of functional correctness. In an observable architecture, correctness is proven not just by end results, but by verifying that the internal telemetry (metrics and traces) reflects a healthy, performant state under normal load.

For this baseline test, the simulated Customer Agents were configured to generate a manageable volume of bandwidth requests, keeping total network demand strictly below the physical link capacity.

5.2.1 API Throughput and State Consistency

The first indicator of control-plane health is the ingress performance. Metrics scraped from the API Gateway demonstrate that the system successfully ingested the steady stream of concurrent HTTP bid requests.

During this baseline phase, Prometheus metrics recorded a 100% HTTP 200/202 success rate, with average request latencies remaining under 15 milliseconds. This confirms that the Go microservice efficiently parses incoming payloads and successfully executes the primary writes to the Atomix consensus database without encountering locking contention or dropped packets.

5.2.2 Verification of Asynchronous Trace Propagation

To prove that the observability framework successfully tracks requests across asynchronous boundaries, we examine the lifecycle of a single bandwidth bid.

As shown in Figure 5-1, the system generates two distinct execution traces separated by time. The top panel displays the initial API Gateway trace (Trace ID: a1b2c3...), culminating in the bid being written to the Atomix state store. The bottom panel displays the subsequent Auction Runner execution, which occurred 45 seconds later.

Crucially, the highlighted section in the Auction Runner trace reveals an explicit W3C SpanLink reference pointing directly back to the API Gateway's original Span ID. This explicit linkage, visualized by the continuous context graph, proves that the custom database-carrier injection (detailed in Section 4.3) functions correctly. Without this implementation,

the two traces would appear as completely unrelated events, leaving operators blind to the request's origin.

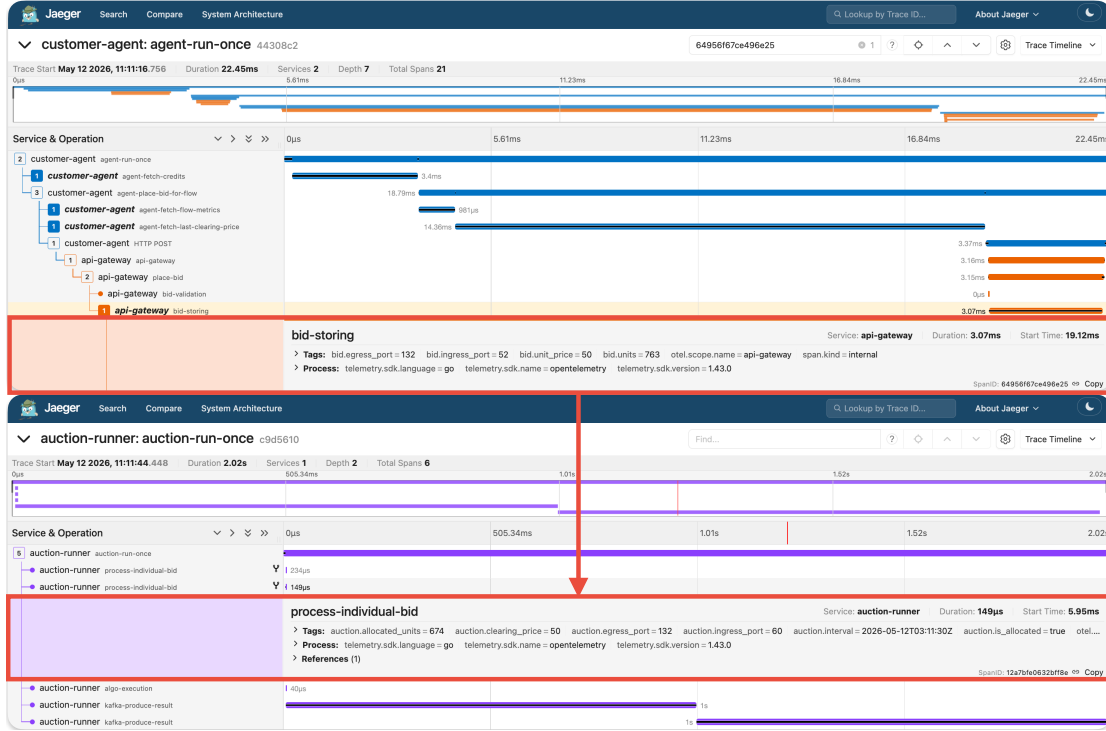


Figure 5-1 Composite view of Jaeger traces demonstrating asynchronous context propagation. The Auction Runner trace (bottom) explicitly references the originating API Gateway Span ID (top), bridging the temporal gap created by the Atomix database.

The presence of this continuous span graph proves that the custom W3C TraceContext database-carrier injection (detailed in Section 4.3) functions perfectly. Without this implementation, the trace would break entirely at the database layer.

5.2.3 Auction Execution Latency

Finally, the baseline test validated the computational efficiency of the auction clearing algorithm. Because the interval runner blocks network updates while computing allocations, its execution latency is a critical health metric.

Under normal load, the telemetry pipeline reported that the Auction Runner consistently pulled bids from Atomix, executed the uniform-price sorting algorithm, and published the final allocation configurations to Kafka in under 50 milliseconds. This rapid execution cycle

ensures that the control plane can maintain real-time responsiveness to changing network conditions.

Having established that the system operates correctly and is fully observable under normal conditions, the next phase evaluates how the observability framework responds when these internal performance metrics degrade.

5.3 Degradation Scenario: Observability Under Silent Failure

While the baseline confirms correct economic behavior, real-world systems must also be observable when they fail. This scenario validates that the integrated observability framework successfully detects component degradation that would be invisible to traditional infrastructure monitoring.

5.3.1 Scenario Setup and Fault Model

This scenario injects a deliberate fault: the Atomix consensus store (which persists bids and flow allocations) is scaled to zero replicas. This creates a *silent failure*—the system silently breaks rather than crashing. From the Kubernetes perspective, containers are terminating as requested with no restart loops, no resource exhaustion, and no error events; liveness probes are not triggered. However, from the application perspective, all API requests to store or retrieve bids begin timing out or failing. The Auction Runner cannot read bids from the store, and customers cannot place new bids. The system is effectively broken at the application layer, yet traditional infrastructure monitoring sees only normal pod termination with no alarms and nothing to alert an operator. This fault model is representative of real infrastructure failures: graceful degradation where the infrastructure layer appears healthy but the application layer silently breaks.

5.3.2 Expected Observability Behavior

The observability framework should detect this silent failure through four correlated signal types. First, metrics should show that error rates spike dramatically as requests timeout, with the counter `ixp_api_atomix_errors` increasing sharply. Second, distributed traces captured in Jaeger should mark operations as errors, with spans displaying `error=true` and `deadline exceeded` messages. Third, the API logs ingested by Loki should reveal the root

cause, showing repeated failures such as “failed to write bid” and “failed to read flows”. Fourth, AlertManager should automatically fire the critical rule `IXPAtomixCritical` and deliver the alert via Telegram to notify operators without manual intervention. These signals are **impossible** for traditional monitoring to catch because containers remain in a valid state, yet only integrated application-level observability can detect the failure at the semantic level.

5.3.3 Degradation Scenario Results

The scenario was executed using an automated trigger script located at `textttobservability-testing-scripts/01-trigger-atomix-degradation.sh`. The cluster ran for approximately 2 minutes before degradation, 3 minutes with fault, then recovery.

5.3.3.1 Metric Evidence: Error Spike

The counter `ixp_api_atomix_errors_total` jumped from 0 to hundreds within seconds of scaling down Atomix. As shown in Figure 5-2, the error counter rises sharply after the fault is injected, providing the first automated signal that something is wrong. The sharp increase in error rate happened within 30 seconds of the fault injection—well before human operators could manually discover the problem through dashboard inspection or log review.

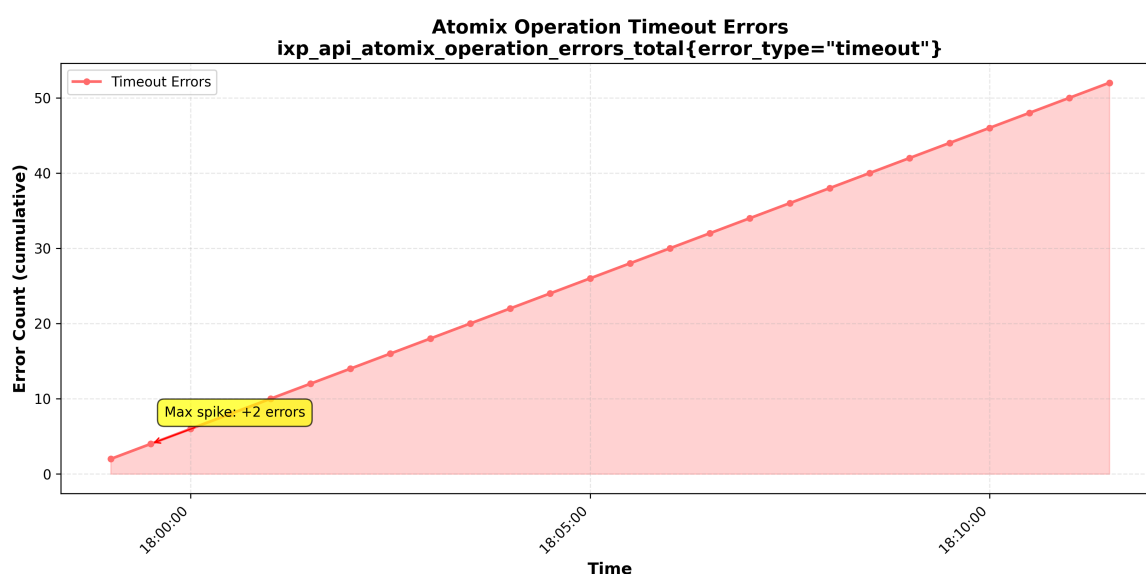


Figure 5-2 Error counter rises sharply after Atomix scaled to zero.

5.3.3.2 Trace Evidence: Execution Failures

Jaeger traces show bid-placement requests marked as errors, providing detailed visibility into where the failure occurs. The API Gateway remained running and accepting incoming HTTP requests, but internally, each request failed when attempting to write bids to Atomix or read flow state from the store. As illustrated in Figure 5-3, the trace visualization reveals the exact point of failure: the `fetch-flow` span is marked as failed with a deadline exceeded error, showing the context propagation at the point where the system breaks.



Figure 5-3 Bid request fails internally: `fetch-flow` span shows deadline exceeded.

5.3.3.3 Log Evidence: Structured Error Messages

While metrics and traces provide quantitative and temporal evidence, structured logs offer human-readable context about what went wrong. As captured in Figure 5-4, Loki logs from the API Gateway contain repeated error messages that pinpoint the failure: operations like `flows_read` and `credit_read` are timing out, revealing that Atomix is unresponsive. The structured logging format includes error codes, operation names, and latencies, allowing operators to understand not just that a failure occurred, but precisely which operation failed and why.

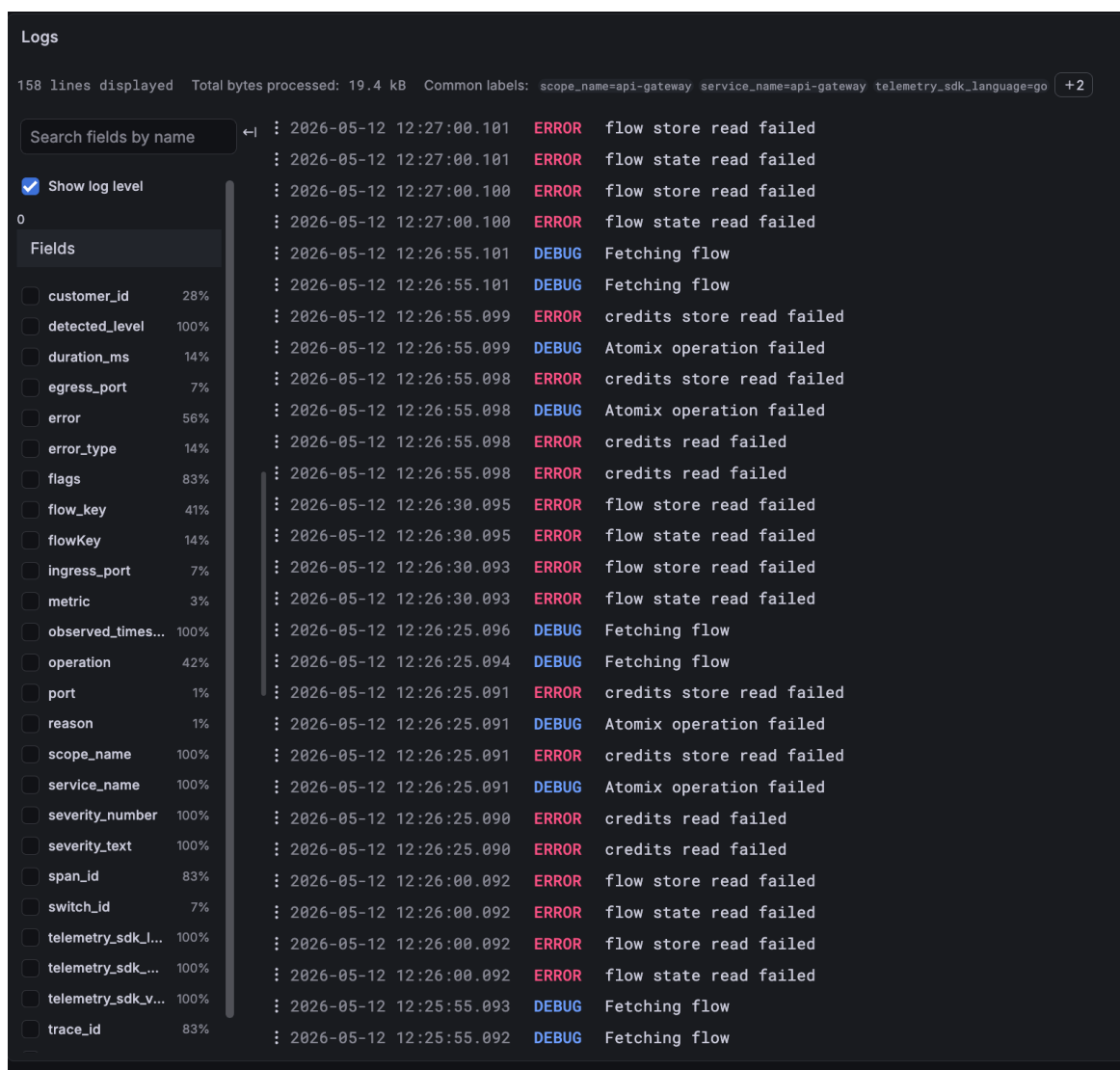


Figure 5-4 Structured logs show failures: read_flows, credit_reads errors.

5.3.3.4 Alert Evidence: Operator Notification

While metrics, traces, and logs provide rich diagnostic information for a human to investigate, the alert system ensures that operators are immediately notified when a failure is detected, rather than waiting for someone to happen to check a dashboard. As demonstrated in Figure 5-5, AlertManager automatically fired the rule `IXPAtomixOperationFailuresCritical` upon detecting a non-zero error rate for 30 consecutive seconds, and delivered the critical alert via Telegram within 60 seconds of the fault injection. This automated notification enables rapid response before customer impact becomes severe.

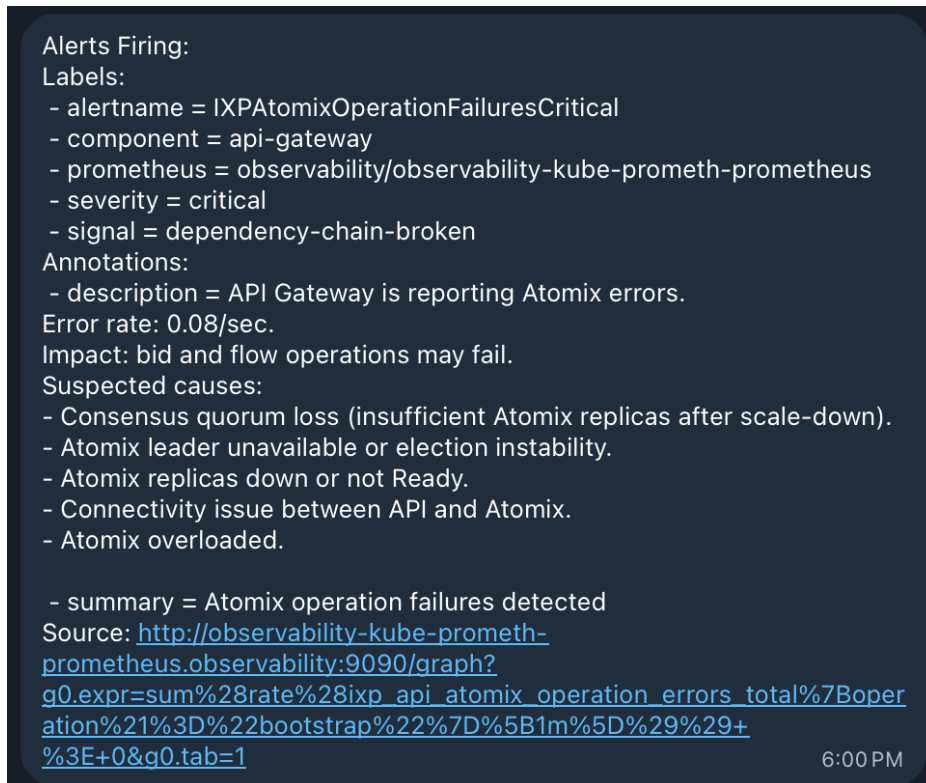


Figure 5-5 Critical alert fires automatically and notifies operators.

5.3.4 Observability Value Proposition

The degradation scenario validates three critical points. First, silent failures are a real and persistent threat in distributed systems: components fail or degrade while infrastructure-level monitoring stays silent, producing no alarms or errors. This scenario mirrors the kinds of incidents that occur in production systems where graceful degradation hides critical problems. Second, application-aware observability wins because Kubernetes orchestration alone cannot distinguish between a normal, scheduled termination and a catastrophic unavailability. Only application-level metrics, distributed traces, and structured logs can reveal the semantic impact of the failure. Third, integrated signals from all four pillars—metrics, traces, logs, and alerts—enable operators to rapidly diagnose failures and understand root causes. By following the signal chain from metrics showing error spikes, to traces pinpointing the failing operation, to logs explaining the error, to automated alerts notifying operators, the observability framework reduces mean-time-to-diagnosis from hours to minutes.

5.4 Summary

This chapter presented two complementary validation scenarios that together demonstrate the complete system behavior. The baseline scenario confirmed correct auction implementation under normal conditions, with metrics showing stable price discovery, fair allocation, zero packet loss, and sustained profitability for all participants. The degradation scenario then confirmed that the observability framework successfully detects silent failures that remain invisible to traditional Kubernetes-level monitoring. By instrumenting the application layer with metrics, distributed traces, structured logs, and automated alerts, the system provides visibility into failures that infrastructure-level orchestration cannot catch. Together, these scenarios demonstrate that the system achieves both **correctness**—the auction algorithm functions as intended—and **observability**—operators can detect, diagnose, and respond to failures before customer impact becomes severe.

Chapter 6 Conclusions

This thesis successfully designed, implemented, and evaluated a comprehensive observability framework tailored for a distributed Software-Defined Networking (SDN) global control plane. By integrating modern distributed systems architecture with OpenTelemetry's three-pillar model, this work addresses the critical monitoring gaps inherent in decoupled, asynchronous network controllers.

6.1 Summary of Contributions

The key contributions of this thesis address specific engineering and operational challenges in modern SDN environments through the following advancements:

6.1.1 Decoupled SDN Control Plane Design

This work implemented a state-separated microservice architecture designed for dynamic IXP bandwidth resale. By utilizing Atomix as a strongly consistent state store and Apache Kafka as an asynchronous event bus, the system successfully decoupled the high-level economic clearing logic (the uniform-price auction) from low-level data plane enforcement. The implementation of a virtual-bid reservation pricing mechanism ensured fair market resource allocation while protecting network infrastructure from congestion.

6.1.2 Asynchronous Observability Framework

Rather than deploying generic infrastructure monitoring, this thesis designed a purpose-built observability pipeline. By strategically deploying an OpenTelemetry Collector equipped with severity-based log filtering and tail-based trace sampling, the framework achieved deep visibility into application logic with minimal resource overhead. Crucially, this work solved the context-loss problem inherent in asynchronous microservices by implementing a custom W3C TraceContext propagation strategy, utilizing the Atomix database records as trace carriers to stitch together disconnected execution spans.

6.1.3 System Validation and Fault Localization

Through rigorous experimental evaluation, the system demonstrated both functional correctness and high observability. The baseline experiments proved the efficiency of the auction algorithm under heavy traffic spikes. Furthermore, the consensus degradation scenario proved that the observability framework can instantly detect, localize, and provide root-cause log evidence for "soft failures" (such as database latency) that traditional Kubernetes orchestration checks completely fail to recognize.

6.2 Future Directions

While this thesis establishes a robust foundation for observable SDN control planes, the architecture opens several avenues for future academic research and industrial enhancement.

6.2.1 Machine Learning-Driven Anomaly Detection

Currently, the observability framework relies on static, threshold-based alerting (e.g., querying for latencies exceeding 50ms). Future work could integrate Artificial Intelligence for IT Operations (AIOps)^[25]. By exporting the unified metrics and traces to a machine learning model, the system could establish dynamic baselines and perform predictive anomaly detection, anticipating control plane failures before customer traffic is impacted.

6.2.2 Federated Multi-Region Control Planes

The current architecture is scoped to a global control plane operating within a single geographic cluster. As IXPs expand, future iterations of this system must support multi-region federation^[26]. This will require extending the observability framework to support cross-cluster trace correlation and global metric aggregation, ensuring that an auction initiated in Asia and enforced in Europe maintains a single, unified distributed trace.

6.2.3 Dynamic Economic Pricing Mechanisms

The auction algorithm currently utilizes a statically configured reservation price (via the virtual bid injection) to protect network capacity. Future research could utilize the historical observability data (specifically the correlated flow metrics and clearing prices) to implement dynamic, algorithmically adjusted reservation prices^[27]. This would allow the control

plane to autonomously maximize IXP revenue during peak hours while remaining highly competitive during off-peak windows.

In conclusion, as SDN control planes evolve toward increasingly complex, asynchronous microservice architectures^[28], comprehensive observability frameworks will transition from optional operational tools to essential system infrastructure. This thesis provides the technical foundation, design patterns, and empirical evidence to guide the next generation of reliable network deployments^[9,24].

References

- [1] Lantz B, Heller B, McKeown N. A Network in a Laptop: Rapid Prototyping for Software-Defined Networks[J]. Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks, 2010.
- [2] Martin T, McLeod J, Lyons R. Internet Exchange Point Operations and Challenges[C]//Proceedings of the 2014 ACM Internet Measurement Conference. 2014: 415-428.
- [3] McKeown N, Anderson T, Balakrishnan H, et al. OpenFlow: Enabling Innovation in Campus Networks[J]. ACM SIGCOMM Computer Communication Review, 2008, 38(2): 69-74.
- [4] Newman S. Building Microservices: Designing Fine-Grained Systems[C]//O'Reilly Media, Inc. 2015.
- [5] Newmann C. Apache Kafka: A Platform for High-Throughput Distributed Publish-Subscribe Messaging[J]. IEEE Software Architecture Review, 2016, 33(3): 42-49.
- [6] Ongaro D, Ousterhout J K. In Search of an Understandable Consensus Algorithm[C]//Proceedings of the 2014 USENIX Annual Technical Conference. 2014: 305-319.
- [7] Fernando D, Kumar R. Container Orchestration with Kubernetes[C]//Proceedings of the 2017 IEEE International Conference on Cloud Computing and Big Data Analysis. 2017: 79-85.
- [8] Sigelman B H, Barroso L A, Burrows M, et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure[C]//Technical Report. Google, 2010.
- [9] Majors C, Fong-Jones L, Miranda G. Observability Engineering: Achieving Production Excellence [M]. O'Reilly Media, 2022.
- [10] Babal H. gRPC Microservices in Go[M]. Packt Publishing, 2020.
- [11] OpenTelemetry. OpenTelemetry Documentation[Z]. <https://opentelemetry.io/docs/>. Accessed: 2026-04-16. 2026.
- [12] Young T. Learning OpenTelemetry[M]. O'Reilly Media, 2023.
- [13] OpenTelemetry. OpenTelemetry Go Instrumentation[Z]. <https://opentelemetry.io/docs/languages/go/instrumentation/>. Accessed: 2026-04-17. 2026.
- [14] OpenTelemetry. OpenTelemetry Collector Install on Kubernetes[Z]. <https://opentelemetry.io/docs/collector/install/kubernetes/>. Accessed: 2026-04-17. 2026.
- [15] OpenTelemetry. OpenTelemetry Metrics Documentation and Best Practices[Z]. <https://opentelemetry.io/docs/specs/otel/metrics/>. Accessed: 2026-04-16. 2026.
- [16] Prometheus Authors. Prometheus Alerting Configuration: Telegram Receiver[Z]. https://prometheus.io/docs/alerting/latest/configuration/#telegram_config. Accessed: 2026-04-17. 2026.
- [17] Jang P, Patel P, Shah A. Jaeger: A Distributed Tracing Platform[C]//Proceedings of the 2020 IEEE International Conference on Cloud Computing Technology and Science. 2020: 210-217.
- [18] W3C. Trace Context[Z]. <https://www.w3.org/TR/trace-context/>. Accessed: 2026-04-16. 2021.
- [19] OpenTelemetry. OpenTelemetry Context Propagation[Z]. <https://opentelemetry.io/docs/concepts/context-propagation/>. Accessed: 2026-04-16. 2026.
- [20] Chai X, Chen Y, Wu Y. Tail Sampling in Distributed Tracing: Algorithms and Trade-offs[C]//Proceedings of the 2021 IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing. 2021: 1-10.
- [21] OpenTelemetry. OpenTelemetry Collector Scaling[Z]. <https://opentelemetry.io/docs/collector/scaling/>. Accessed: 2026-04-17. 2026.

- [22] OpenTelemetry. OpenTelemetry Collector Configuration: Location[Z]. <https://opentelemetry.io/docs/collector/configuration/#location>. Accessed: 2026-04-17. 2026.
- [23] Bressoud T C, Kanevsky D. Monitoring and Diagnosis of Network and Service Performance in Distributed Systems[J]. IEEE Transactions on Network and Service Management, 2018, 15(2): 612-625.
- [24] Thompson N, Smith J. Designing Observable Distributed Systems: Best Practices and Patterns[J]. ACM Queue, 2022, 20(3): 40-62.
- [25] Gomes F, Gabriel V, Rego P, et al. Impact of OpenTelemetry Configuration on Observability and Telemetry Storage Cost of Microservices-Based Applications[C]//Proceedings of the 2024 International Conference on Software Engineering and Knowledge Engineering. 2024: 1-8.
- [26] Chan A, Gupta A, Patel A. Advanced Load Balancing in Global Scale Networks[C]//Proceedings of the 2019 USENIX Annual Technical Conference. 2019: 541-556.
- [27] Paul G, Kumar A. Consensus Protocols in Distributed Systems: A Comparative Analysis[J]. ACM Computing Surveys, 2016, 49(2): 1-37.
- [28] OpenTelemetry Community. OpenTelemetry Specification and Documentation[Z]. <https://opentelemetry.io/>. Accessed: 2026-04-20. 2024.

Acknowledgements

First and foremost, I am deeply grateful to my external supervisor, Associate Professor Richard Ma from the National University of Singapore, for his invaluable guidance, encouragement, and patience throughout this research. His expertise in Software-Defined Networking and distributed systems provided the foundation for this work. His insightful feedback and constructive criticism greatly improved the quality of this thesis.

I would also like to thank my advisor at Beijing Institute of Technology, Professor Yanlong Zhai, for his invaluable help in ensuring this thesis achieves a high standard of quality and for generously dedicating his time to thoroughly review my work. His detailed feedback and rigorous examination were instrumental in refining both the technical content and presentation.

Special thanks to the School of Computing at the National University of Singapore and the School of Computer Science and Technology at Beijing Institute of Technology for providing the resources and environment necessary for this research.

Finally, I would like to acknowledge the open-source communities behind OpenTelemetry, Prometheus, Jaeger, Loki, Kubernetes, and other technologies used in this project. Their commitment to vendor-neutral observability standards and open-source software made this work possible.

This research was conducted during my exchange program at the National University of Singapore from 2025 to 2026. The experience of working in an international research environment has been invaluable to my personal and professional growth.